

CBO Academy - CAP Initiative

Technical Design Document

<Medical Expense Summary>

Version 1.0

12/12/2024

Revision History

DATE	VERSION	REVISION DESCRIPTION	Creator	Reviewer
12/12/2024	1.0.0	First Draft of Medical Summary Screen	Kshitij Pandey	Reshmi

Contents

Revision History	2
1. Introduction	4
1.1. Scope	4
1.2. Assumptions	4
1.3. References	4
1.4. Acronyms	4
2. <Medical Expense Summary>	5
2.1. Component Diagram	5
2.2. Key Classes	5
2.3. Class Diagram	5
2.4. API Endpoint Details	5
2.5. API Response Codes	5
2.6. <Medical Expense Summary> API Details	6
2.6.1. API Structure	6
2.6.2. Sequence Diagram	7
2.6.3. Request Processing Steps	7
2.6.4. Request Validation Rules	7
2.6.5. Request/Response Data Mapping	7
2.6.6. Native Query	7
3. Configuration	8
3.1. <propertyfilename>.properties	8
4. Additional Details	9
4.1. Exception Handling	9
4.2. Health Check Configuration	9
4.3. Swagger Configuration	9
4.4. GIT Repo Details	9

1. Introduction

1.1. Scope

The Medical Expense Summary system is designed to provide a comprehensive and user-friendly solution for managing and analyzing medical expenses. By incorporating features such as detailed data management, a robust user interface, data export functionality, and specific user actions, the system ensures that users can efficiently record, edit, delete, and analyze their medical expenses.

1.2. Assumptions

1. **User Authentication:** It is assumed that the user is already logged in with correct credentials.
2. **Single Page Application (SPA):** This implies that the entire application is loaded once, and subsequent interactions and data updates are managed dynamically without requiring full page reloads.

1.3. References

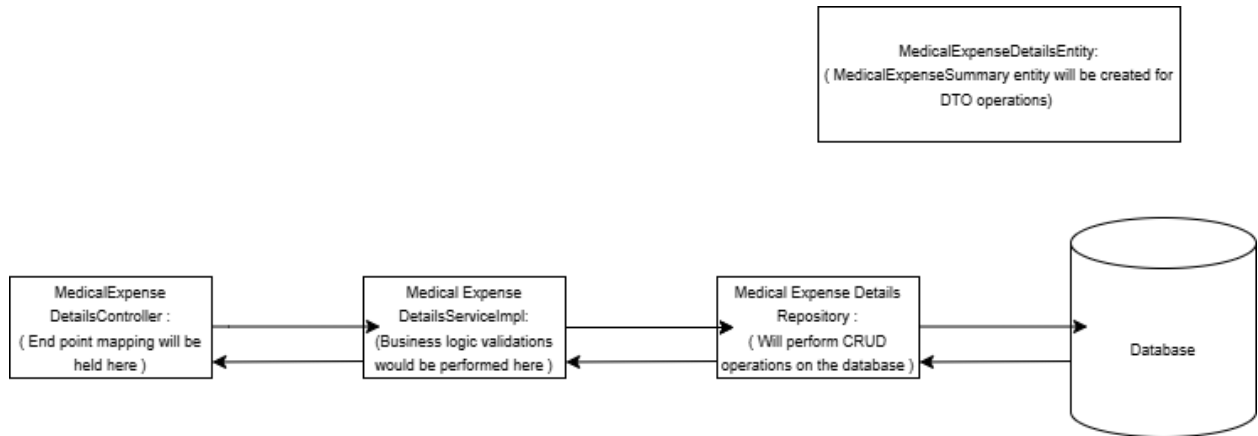
No.	Reference Document	Location/Path
1.	MedicalExpense_v1.xlsx	Local Storage

1.4. Acronyms

Acronyms	Description
DB	Database
DTO	Data Transfer Object

2. < Medical Expense Summary >

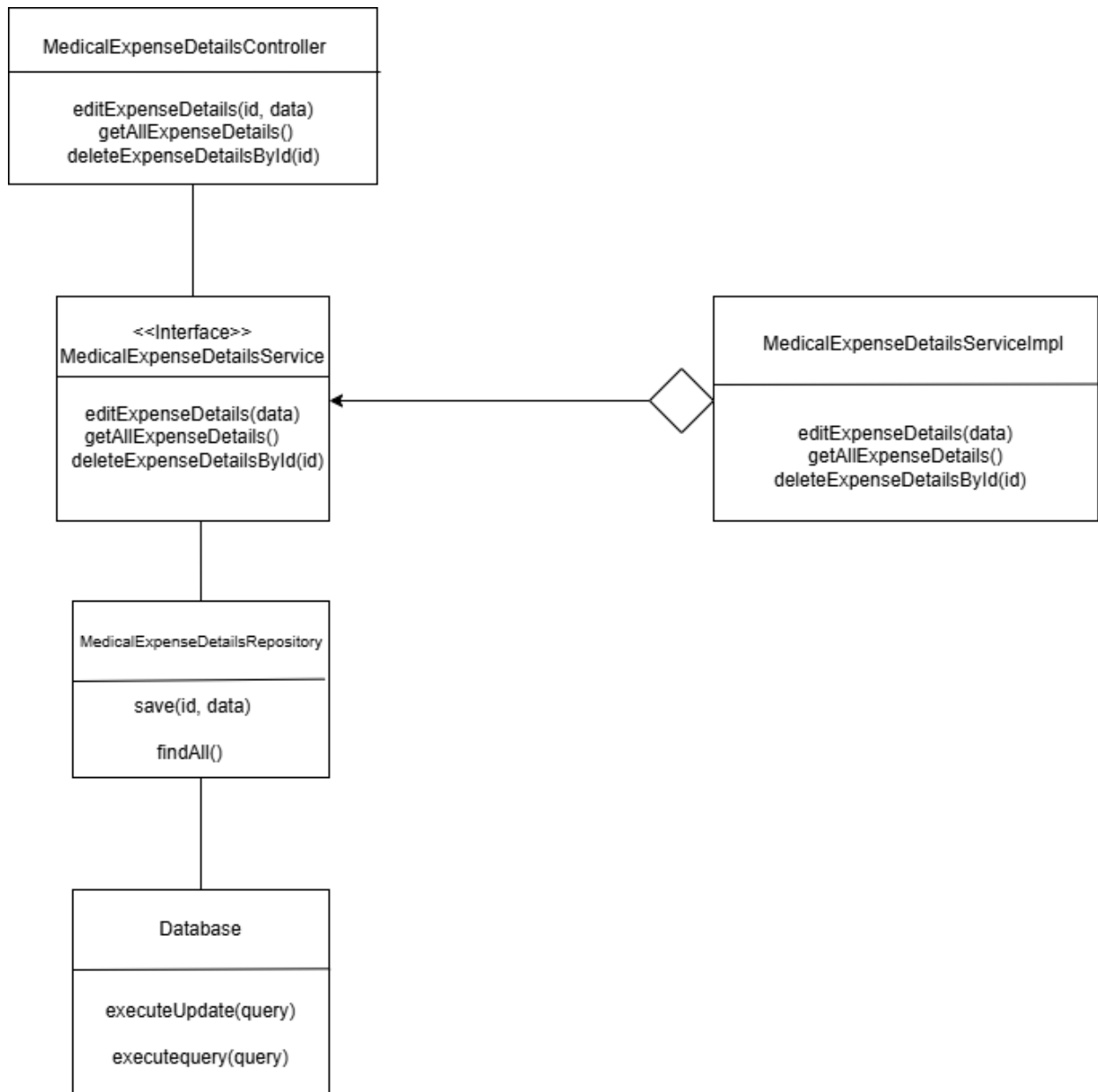
2.1. Component Diagram



2.2. Key Classes

Class Name	Package Name	Description
MedicalExpenseDetailsService.java	com.deloitte.nextgen.appreg.web	The interface defines the business logic for managing medical expense summaries. It declares methods that the service implementation class must provide. This interface ensures that the business logic is abstracted from the controller and can be easily tested and maintained.
MedicalExpenseDetailsServiceImpl.java	com.deloitte.nextgen.appreg.web	The class is the implementation of the MedicalExpenseSummaryService interface. It contains the actual business logic for managing medical expense summaries. This class interacts with the repository layer to perform CRUD operations on the database.
MedicalExpenseDetailsRepository.java	com.deloitte.nextgen.appreg.web	The interface is responsible for data access operations related to medical expense summaries. It extends a Spring Data repository interface (e.g., JpaRepository, CrudRepository) to provide CRUD operations without the need for boilerplate code.
MedicalExpenseDetails.java	com.deloitte.nextgen.appreg.web	The class is an entity that represents a medical expense summary in the database. It is mapped to a database table using JPA annotations. This class typically includes fields for the properties of a medical expense summary, along with getters and setters.
MedicalExpenseDetailsController.java	com.deloitte.nextgen.appreg.web	This class is responsible for handling HTTP requests related to medical expense summaries. It acts as an intermediary between the client (e.g., a web browser or mobile app) and the service layer. This controller typically includes endpoints for creating, reading, updating, and deleting medical expense summaries.

2.3. Class Diagram



2.4. API Endpoint Details

HTTP Method	Endpoint Path	Description
GET	/medicalexpensesummary	This endpoint fetches all medical expense summary records. It retrieves the data object from the medical expense details.
DELETE	/medicalexpensesummary/{id}	This endpoint deletes a specific medical expense summary based on the provided ID. It removes the corresponding record from the medical expense details.
PUT	/medicalexpensesummary/{id}	This endpoint edits a specific medical expense summary based on the provided ID. It updates the corresponding record in the medical expense details with the new data provided in the request body.

2.5. API Response Codes

Response Code	Description
200 – OK	The request was successful, and the server returned the requested data or confirmed the successful execution of the requested operation.
500 – Internal Server Error	The server encountered an unexpected condition that prevented it from fulfilling the request.
503 – Unavailable	The server is currently unable to handle the request due to temporary overloading or maintenance of the server.
201 - Created	The HTTP 201 Created status code indicates that the request has been successfully fulfilled and has resulted in the creation of a new resource. This response code is typically returned by the server after a POST request that creates a new resource on the server.

2.6. < Medical Expense Summary > API Details

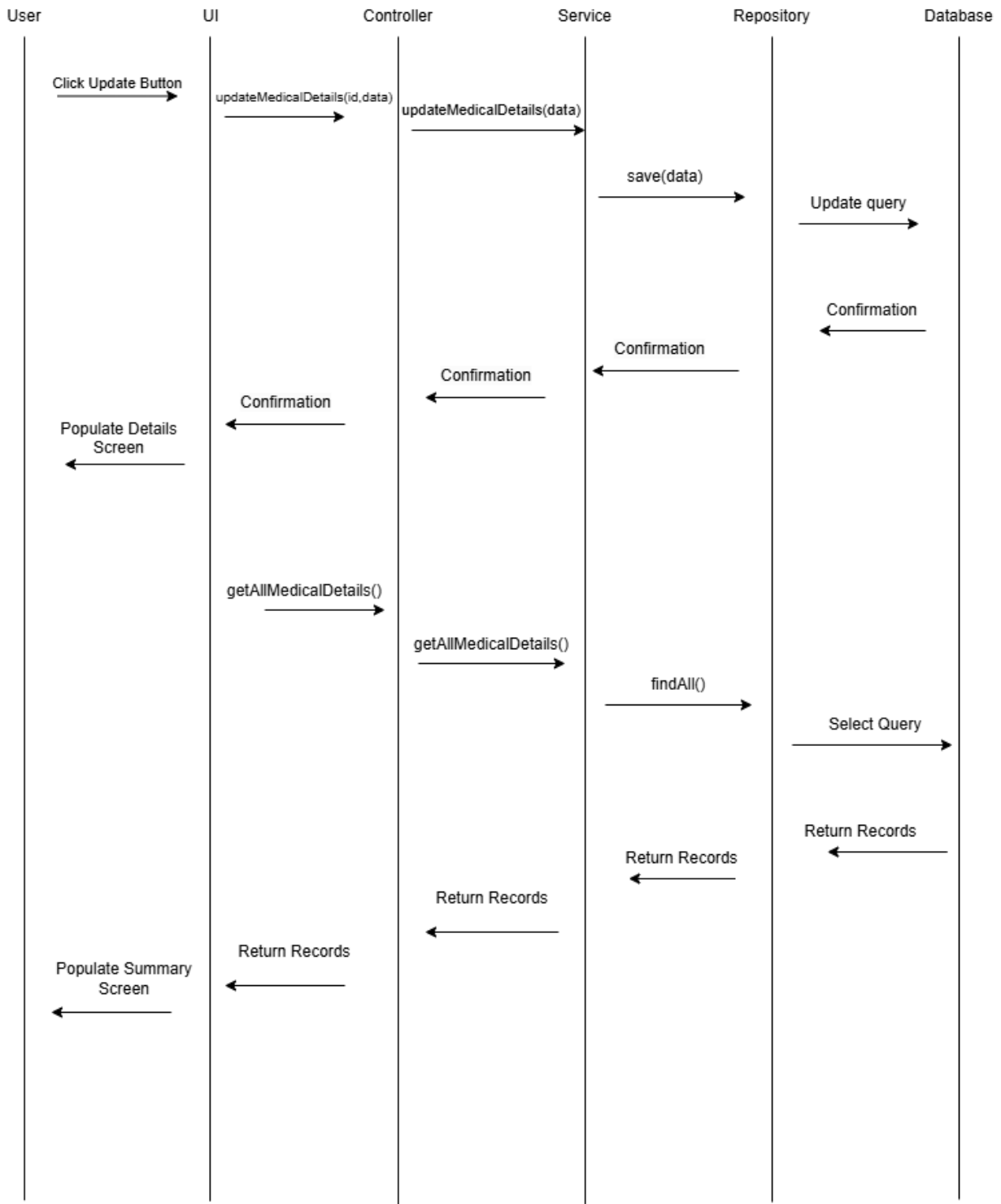
2.6.1. API Structure

Topic	Description
Overview	
URL	POST/XXX/XXXX
Request Headers	Content-Type: application/json X-AUTH-TOKEN <ENCRYPTED_JWT_TOKEN>

Type	JSON Parameters
Sample Request	Request Headers: Request Payload: GET: MedicalExpenseSummary JSON Object List DELETE: MedicalExpenseSummary JSON Object ID: Id of the currently logged in user LEGEND: • Values with font color in Blue – Optional • Values with font color in Black – Mandatory
Sample Response (Response Status Code: 200)	<pre>[{ "medicalId": 129, "responsiblePersonId": 1234567891011, "responsibleForExpense": "Pratik", "expenseIncurredId": 1234576891014, "expenseIncurred": "Kshitij", "outsideHousehold": "t", "changeDate": "2025-01-11", "endDate": "2025-01-12", "reportedOn": "2025-01-12", "expenseType": "AM", "expenseSubType": "PS", "expensePaidTo": "129", "expensePaidToId": 1234576891014, "personOutsideHousehold": "KP", "incurredDate": "2025-01-12", "frequency": "DA", "amountDueFrequency": 100.0, "medicalExpenseReimbursed": 100.0, "finalExpenseDate": "2025-01-12", "verification": "TP",</pre>

	<pre> "comments": "comment", "householdIncurMedicalCost": "Y", "typeOfTransportation": "Sk", "placeOfVisit": "Drug", "milesTraveled": "12", "numberOfVisits": "12", "payPerVisit": 12.0, "tripsPerMonth": "132", "outsideHouseholdPay": "Y", "giveMoneyToHouseholdMember": "Y", "helpPayExpense": "KP", "howMuchExpense": 12.0, "howOftenExpense": "12" }] </pre>
Error Sample Response (Response Status Code: 500)	<pre> { "error": "Internal Server Error", "message": "An unexpected error occurred. Please try again later." } </pre>
Error Sample Response (Response Status Code: 503)	<pre> { "error": "Service Unavailable", "message": "The server is currently unable to handle the request due to temporary overloading or maintenance. Please try again later." } </pre>

2.6.2. Sequence Diagram



2.6.3. Request Processing Steps

Processing Step	Description
Step 1	Validate Request: The server validates the incoming request headers and payload to ensure they are correctly formatted and contain all required information. This includes checking the Content-Type and X-AUTH-TOKEN headers, as well as verifying the presence and validity of mandatory fields in the JSON payload. For example, it checks if the medical expense summary details such as patientID, expenseDate, and amount are provided and correctly formatted.
Step 2	Process Request: The server processes the request by performing the necessary operations. This could involve interacting with the database to create, read, update, or delete medical expense summary records based on the request type (e.g., POST, GET, PUT, DELETE). During this step, business logic is applied to ensure the operation is performed correctly. For instance, if it is a POST request, the server will add a new medical expense summary record to the database.
Step 3	Generate Response: The server generates an appropriate response based on the outcome of the request processing. If the request was successful, a 200 status code and the relevant data are returned. If an error occurred, an appropriate error status code (e.g., 500, 503) and error message are returned. The response is then sent back to the client. For example, if the medical expense summary was successfully created, the server returns a 200 status code along with the details of the newly created record.

2.6.4. Request/Response Data Mapping

Request/Response Element	Table Name	Column Name	Data Type	Sample Values
Person Responsible for paying expense	DC_MEDICAL_DEDUCTIONS	NAME	VARCHAR(100)	“Chitrali”
Person for Whom the Expense is Incurred	DC_MEDICAL_DEDUCTIONS	DEP_NAME	VARCHAR(100)	“Kshitij”
Incurred Date	DC_MEDICAL_DEDUCTIONS	SERVICE_DT	DATE	01-15-2025
Type	DC_MEDICAL_DEDUCTIONS	MED_EXP_TYPE_CD	VARCHAR(5)	Surgery
Frequency	DC_MEDICAL_DEDUCTIONS	FREQUENCY	VARCHAR(5)	Daily
Amount Due/Freq	DC_MEDICAL_DEDUCTIONS	AMT_PAID	NUMBER	101

2.6.5. Database Query

-- Delete a specific record from the medical_deduction table

```
DELETE FROM wp_cap.medical_deduction WHERE deduction_seq_num = 29;
```

-- Select all records from the medical_deduction table

```
SELECT * FROM wp_cap.medical_deduction;
```

3. Configuration

The following project specific properties are used and stored in properties file.

3.1. <propertyfilename>.properties

Key	Description	Examples
server.servlet.contextPath	The prefix of a URL path that is used to select the context to which an incoming request is passed	server.servlet.contextPath= /
server.port	Port number on which Microservice will be available. If the application is deployed in Container, it will use the server's managed server port.	server.port= 2611
management.contextPath	This is to group all management endpoints of actuator under a single path	management.contextPath= /medicalexpensesummary/

4. Additional Details

4.1. Exception Handling

4.2. Health Check Configuration

Type	URL	Response
GET	localhost:2611/deg/angular/p5/medicalExpense	[{ "medicalId": 129, "responsiblePersonId": 1234567891011, "responsibleForExpense": "Pratik", "expenseIncurredId": 1234576891014, "expenseIncurred": "Kshitij", "outsideHousehold": "t", "changeDate": "2025-01-11", "endDate": "2025-01-12", "reportedOn": "2025-01-12", "expenseType": "AM", "expenseSubType": "PS",

		<pre> "expensePaidTo": "129", "expensePaidTold": 1234576891014, "personOutsideHousehold": "KP", "incurredDate": "2025-01-12", "frequency": "DA", "amountDueFrequency": 100.0, "medicalExpenseReimbursed": 100.0, "finalExpenseDate": "2025-01- 12", "verification": "TP", "comments": "comment", "householdIncurMedicalCost": "Y", "typeOfTransportation": "Sk", "placeOfVisit": "Drug", "milesTraveled": "12", "numberOfVisits": "12", "payPerVisit": 12.0, "tripsPerMonth": "132", "outsideHouseholdPay": "Y", "giveMoneyToHouseholdMember": "Y", "helpPayExpense": "KP", "howMuchExpense": 12.0, "howOftenExpense": "12" }] </pre>
DELETE	localhost:2611/deg/angular/p5/medicalExpense/8	<pre> { "status": "Deleted Record Successfully." } </pre>

4.3. GIT Repo Details

The source code for < **angular-deg-fe** > can be accessed in Git repo using the below link:

Project Name: **angular-deg-fe**

HTTPS URL: <https://github.com/Deloitte-US/angular-deg-fe/>